

Day 16: Encapsulation

- Access modifiers
- Getters and setters
- Data hiding

1. What is Encapsulation?

Simple definition:

Encapsulation means wrapping data and the methods that operate on that data into a single unit, usually a class, and controlling how that data can be accessed.

This sounds complicated, so let's break it.

Suppose we have a student.

A student has data:

name

age

marks

And actions:

study()

writeExam()

displayDetails()

We put related data and methods together inside one class:

```
class Student {
```

```
    String name;
```

```
    int age;
```

```
    int marks;
```

```
    void study() {
```

```

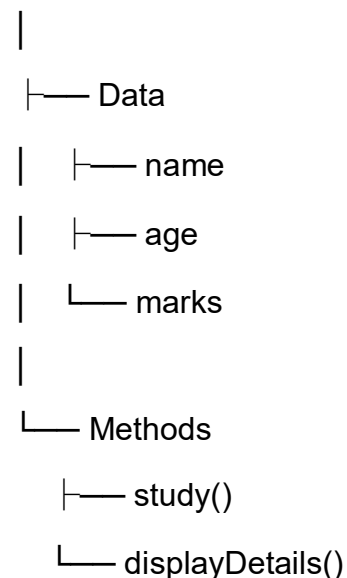
        System.out.println("Student is studying");
    }

    void displayDetails() {
        System.out.println(name);
        System.out.println(age);
        System.out.println(marks);
    }
}

```

Here:

Student



This **bundling/wrapping of data and methods inside a class** is one important part of encapsulation.

But Java's encapsulation is not just "put everything inside a class."

The more important part is:

We control how outside code can access the data.

For example, should anybody be allowed to directly change a student's marks?

Maybe not.

We can prevent direct access and provide controlled methods instead.

That's where:

- private
- getters
- setters

come into the picture.

2. Why do we need Encapsulation?

Imagine this class:

```
class BankAccount {
```

```
    String accountHolder;
```

```
    double balance;
```

```
}
```

And somewhere else:

```
BankAccount account = new BankAccount();
```

```
account.balance = -50000;
```

Java allows this because balance is accessible.

But logically, a bank account shouldn't have:

```
balance = -50000
```

as a normal balance value.

The problem is:

Outside code has unrestricted access to our data.

Encapsulation helps us control this.

Instead of:

```
account.balance = -50000;
```

we can make:

```
private double balance;
```

Now outside code cannot directly access it.

Then we provide methods:

deposit()

withdraw()

getBalance()

These methods control what happens to the data.

For example:

```
class BankAccount {  
  
    private double balance;  
  
    void deposit(double amount) {  
        if (amount > 0) {  
            balance = balance + amount;  
        }  
    }  
  
    double getBalance() {  
        return balance;  
    }  
}
```

Now:

```
BankAccount account = new BankAccount();
```

```
account.deposit(5000);
```

```
System.out.println(account.getBalance());
```

The user cannot directly do:

```
account.balance = -50000;
```

because balance is private.

This is the core idea of encapsulation.

3. Real-life example

Think about a **washing machine**.

Inside the washing machine there are many things:

motor

wires

water system

drum

sensors

controller

Do you directly operate the motor?

No.

You interact with:

Power button

Start button

Mode button

Temperature button

You don't need direct access to the internal components.

You say:

Start

and the washing machine internally performs many operations.

Similarly in Java:

Outside world

↓

Public methods

↓

Private data

The outside world doesn't directly manipulate the internal data.

4. Encapsulation structure

A common Java encapsulation pattern is:

```
class Student {  
  
    private String name;  
    private int age;  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    public int getAge() {  
        return age;  
    }  
  
    public void setAge(int age) {  
        this.age = age;  
    }  
}
```

Here:

private variables



Getters / Setters



Outside code

This is the pattern you will see very frequently in Java.

5. Access Modifiers

Now let's understand the first major topic.

What is an Access Modifier?

An access modifier tells Java:

Who is allowed to access this class, variable, method, or constructor?

In simple words:

Access modifier = permission

Like real life:

Private room

Public road

Restricted area

Different things have different access permissions.

Java has four important access levels:

public

protected

default

private

Let's understand each carefully.

6. public

public means:

Accessible from anywhere where the class/member is accessible.

Example:

```
class Student {
```

```
    public String name;
```

```
public void study() {  
    System.out.println("Studying");  
}  
}
```

Outside:

```
Student s = new Student();
```

```
s.name = "Ravi";
```

```
s.study();
```

This is allowed.

Because:

```
public
```

means broad access.

Think:

PUBLIC = "Everyone can access"

7. private

This is extremely important for encapsulation.

private means:

Accessible only inside the same class.

Example:

```
class Student {
```

```
    private int marks;
```

```
    void displayMarks() {
```

```
        System.out.println(marks);
```

```
    }
```

```
}
```


Now:

```
Student s = new Student();
```

```
s.marks = 90;
```

This gives an error.

Why?

Because marks is:

```
private
```

and we are trying to access it outside Student.

But this is allowed:

```
class Student {
```

```
    private int marks;
```

```
    void displayMarks() {
```

```
        System.out.println(marks);
```

```
    }
```

```
}
```

Why?

Because displayMarks() is inside the same class.

So:

Student class

```
|
```

```
|— marks    ← private
```

```
|
```

```
|— displayMarks ← can access marks
```

Remember:

private = only inside the same class

8. Why do we make variables private?

This is one of the most important questions.

Suppose:

```
class Student {
```

```
    public int marks;
```

```
}
```

Outside:

```
Student s = new Student();
```

```
s.marks = -100;
```

Java allows it.

But logically:

```
marks = -100
```

may not make sense.

Instead:

```
class Student {
```

```
    private int marks;
```

```
    public void setMarks(int marks) {
```

```
        if (marks >= 0 && marks <= 100) {
```

```
            this.marks = marks;
```

```
        }
```

```
    }
```

```
}
```

Now:

```
s.setMarks(90);
```

works.

But:

```
s.setMarks(-100);
```

doesn't modify the value.

So we have **control**.

This is why encapsulation is useful.

9. protected

protected is slightly more advanced.

It allows access:

- inside the same class
- inside the same package
- through inheritance in other packages

For now, remember the basic idea:

```
protected int age;
```

is more accessible than private, but less accessible than public.

Since you're learning inheritance, this becomes important.

Example:

```
class Animal {
```

```
    protected String name;
```

```
}
```

A child class can access it:

```
class Dog extends Animal {
```

```
    void display() {
```

```
        System.out.println(name);
```

```
    }
```

```
}
```

Because Dog inherits from Animal.

10. Default access modifier

There is something interesting in Java.

If you don't write any access modifier:

```
class Student {
```

```
    int marks;
```

```
}
```

then marks has **default/package-private access**.

That means it can generally be accessed by classes in the **same package**, but not from unrelated classes in another package.

Example:

```
int marks;
```

is different from:

```
private int marks;
```

and different from:

```
public int marks;
```

11. Access Modifier Comparison

Think of them like levels of permission:

Modifier	Same Class	Same Package	Subclass	Other Package
private	✓	✗	✗*	✗
default	✓	✓	✓ if same package	✗
protected	✓	✓	✓	Limited through inheritance
public	✓	✓	✓	✓

* A subclass cannot directly access a private member as an inherited accessible member. It can interact with it through methods such as getters/setters.

For beginner-level understanding:

private

↓

default

↓

protected

↓

public

As access becomes broader.

12. Which access modifier is commonly used for encapsulation?

Usually:

private

for data.

And:

public

for controlled methods.

For example:

```
class Student {
```

```
    private int marks;
```

```
    public int getMarks() {
```

```
        return marks;
```

```
    }
```

```
    public void setMarks(int marks) {
```

```
        this.marks = marks;
    }
}
```

Notice:

```
private int marks;
```

but:

```
public int getMarks()
```

and:

```
public void setMarks()
```

Why?

Because we want:

marks → protected

methods → accessible

The outside world can't directly touch the variable.

It has to go through the methods.

13. Getters and Setters

Now let's understand the second major topic.

Suppose:

```
class Student {
```

```
    private String name;
```

```
    private int age;
```

```
}
```

Because these variables are private:

```
Student s = new Student();
```

```
System.out.println(s.name);
```

is not allowed.

Then how can we **read** name?

We use a **getter**.

14. What is a Getter?

A getter is a method used to **get/read** the value of a private variable.

Example:

```
public String getName() {  
    return name;  
}
```

Let's break it:

public

→ other classes can call it.

String

→ method returns a String.

getName()

→ method name.

return name;

→ gives the private variable's value back.

So:

String name = s.getName();

means:

Outside

↓

getName()

↓

private name

↓

value returned

15. Why is it called Getter?

Because:

get = obtain/read

Examples:

getName()

getAge()

getMarks()

getSalary()

getBalance()

Usually getters start with:

get

16. Setter

Now suppose we want to **change** the value.

We can't do:

s.age = 20;

if age is private.

So we use a setter.

Example:

```
public void setAge(int age) {  
    this.age = age;  
}
```

A setter is a method used to **set/change** the value of a private variable.

17. Getter vs Setter

Very simple:

GETTER → reads value

SETTER → changes value

Example:

getAge()

means:

"Give me the age."

While:

setAge(20)

means:

"Set the age to 20."

18. Complete Example

```
class Student {
```

```
    private String name;
```

```
    private int age;
```

```
    public String getName() {
```

```
        return name;
```

```
    }
```

```
    public void setName(String name) {
```

```
        this.name = name;
```

```
    }
```

```
    public int getAge() {
```

```
        return age;
```

```
    }
```

```
    public void setAge(int age) {
```

```
        this.age = age;
```

```
    }
```

```
}
```

Now:

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Student s = new Student();
```

```
        s.setName("Ravi");
```

```
        s.setAge(20);
```

```
        System.out.println(s.getName());
```

```
        System.out.println(s.getAge());
```

```
    }
```

```
}
```

Output:

Ravi

20

19. Understand the flow carefully

When we write:

```
s.setName("Ravi");
```

we are **not directly changing name from outside**.

Instead:

Main

↓

```
setName("Ravi")
```

↓

Student object

↓

`this.name = name`

↓

name becomes "Ravi"

Similarly:

`s.getName();`

means:

Main

↓

`getName()`

↓

return name

↓

Main receives "Ravi"

20. Why do we need Getter and Setter?

You may ask:

"If I can directly access the variable, why create another method?"

Excellent question.

Suppose:

`public int age;`

Then anyone can do:

`s.age = -500;`

No control.

But:

`private int age;`

with:

`public void setAge(int age) {`

```
    if (age >= 0) {  
        this.age = age;  
    }  
}
```

Now:

```
s.setAge(-500);
```

doesn't allow invalid data to be stored.

So getter/setter provides a **controlled gateway**.

21. Setter with Validation

This is where encapsulation becomes really powerful.

```
class Student {  
  
    private int marks;  
  
    public void setMarks(int marks) {  
  
        if (marks >= 0 && marks <= 100) {  
            this.marks = marks;  
        } else {  
            System.out.println("Invalid marks");  
        }  
    }  
  
    public int getMarks() {  
        return marks;  
    }  
}
```

Now:

```
Student s = new Student();
```

```
s.setMarks(85);
```

works.

But:

```
s.setMarks(150);
```

prints:

Invalid marks

The internal data remains protected.

22. This is the real power of Setter

A setter doesn't have to simply do:

```
this.age = age;
```

It can perform logic.

For example:

```
public void setAge(int age) {
```

```
    if (age >= 0 && age <= 120) {
```

```
        this.age = age;
```

```
    }
```

```
}
```

Or:

```
public void setSalary(double salary) {
```

```
    if (salary > 0) {
```

```
        this.salary = salary;
```

```
    }
```

```
}
```

Or:

```
public void setPassword(String password) {  
  
    if (password.length() >= 8) {  
        this.password = password;  
    }  
}
```

So the setter can act as a **security/validation checkpoint**.

23. What is Data Hiding?

Now the third major topic.

Data hiding means:

Preventing direct access to the internal data of an object from outside the class.

Usually we achieve this using:

private

Example:

```
class BankAccount {  
  
    private double balance;  
}
```

Now outside code cannot directly access:

account.balance

The internal data is hidden.

24. Data hiding ≠ Encapsulation

This distinction is important.

They are related, but not exactly the same.

Data hiding

Focus:

Prevent direct access to internal data.

Example:

```
private int marks;
```

Encapsulation

Focus:

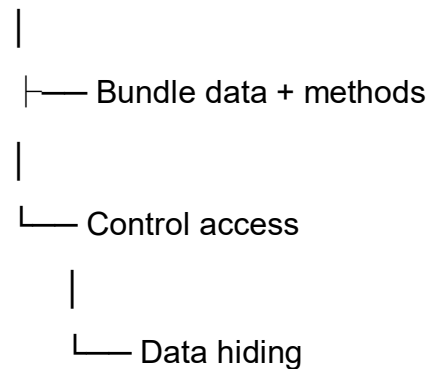
Bundling data + methods together and controlling access to the data.

Example:

```
class Student {  
  
    private int marks;  
  
    public void setMarks(int marks) {  
        ...  
    }  
  
    public int getMarks() {  
        ...  
    }  
}
```

So:

Encapsulation



Data hiding is an important part of achieving encapsulation.

25. Very important distinction

Don't memorize this as:

"Encapsulation means private variables."

That is incomplete.

Instead remember:

Encapsulation is the concept of wrapping data and related behavior together and controlling access to the internal state.

And commonly:

private data

+

public getter/setter/methods

is used to implement it.

26. Why not make everything public?

Suppose:

```
class BankAccount {
```

```
    public double balance;
```

```
}
```

Then:

```
account.balance = -999999;
```

Anyone can modify it.

Problems:

No validation

No control

No protection

Hard to maintain

Internal state exposed

Instead:


```
class BankAccount {  
  
    private double balance;  
  
    public void deposit(double amount) {  
  
        if (amount > 0) {  
            balance += amount;  
        }  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
}
```

Now the object controls its own state.

27. Does every private variable need a Getter AND Setter?

No.

This is very important.

Beginners often think:

private variable

↓

getter compulsory

↓

setter compulsory

No.

You decide what access is required.

Case 1: Getter only

Suppose a student's ID should not change after creation.

```
class Student {  
  
    private int id;  
  
    public int getId() {  
        return id;  
    }  
}
```

There is no:

setId()

So outside code can **read** the ID but cannot change it through a setter.

Case 2: Setter only

Rare, but possible.

For example, a class may allow something to be updated but not read directly.

```
private String password;  
  
public void setPassword(String password) {  
    this.password = password;  
}
```

No getter.

Case 3: Getter + Setter

Normal editable data:

```
private String name;  
  
public String getName() {
```

```
    return name;
}

public void setName(String name) {
    this.name = name;
}
```

Case 4: Neither

Sometimes data is completely internal.

```
private int calculationResult;
```

No getter/setter required if outside code doesn't need access.

28. Read-only and Write-only concepts

This follows naturally.

Read-only

Getter but no setter:

```
private int id;
```

```
public int getId() {
    return id;
}
```

Outside:

```
System.out.println(s.getId());
```

Allowed.

But:

```
s.setId(10);
```

doesn't exist.

So:

READ ☒

WRITE ✕

Write-only

Setter but no getter:

```
private String password;
```

```
public void setPassword(String password) {  
    this.password = password;  
}
```

So:

WRITE ✓

READ ✕

This is useful when you don't want to expose the actual stored value.

29. Why do we use this in setters?

You already learned this, so let's connect it.

Consider:

```
class Student {  
  
    private String name;  
  
    public void setName(String name) {  
        this.name = name;  
    }  
}
```

There are two names:

```
private String name;
```

and:

```
String name
```

The first:

`this.name`

means:

object's instance variable

The second:

`name`

means:

method parameter

So:

`this.name = name;`

means:

object's name = parameter name

Example:

`s.setName("Ravi");`

Inside:

`this.name = name;`

becomes conceptually:

student object's name = "Ravi"

30. What if we don't use this?

Suppose:

```
class Student {
```

```
    private String name;
```

```
    public void setName(String name) {
```

```
        name = name;
```

```
    }
```

```
}
```

This is wrong logically.

Both sides refer to the parameter:

name = name

You're basically saying:

parameter = parameter

The object's variable doesn't get updated.

That's why:

this.name = name;

is used.

31. Encapsulation with Constructor

You can combine constructors with encapsulation.

```
class Student {
```

```
    private String name;
```

```
    private int age;
```

```
    Student(String name, int age) {
```

```
        this.name = name;
```

```
        this.age = age;
```

```
    }
```

```
    public String getName() {
```

```
        return name;
```

```
    }
```

```
    public int getAge() {
```

```
        return age;
```

```
    }
```

```
}
```

Now:

```
Student s = new Student("Ravi", 20);
```

The values are initialized through the constructor.

Notice something interesting:

There are no setters.

Why?

Because maybe we don't want the values to be changed later.

So:

Constructor

↓

Initializes data

Getter

↓

Reads data

No Setter

↓

Data cannot be changed through setter

This is another way to design a class.

32. Encapsulation + Inheritance

Let's connect today's concept with your previous topic.

Parent:

```
class Animal {
```

```
    private String name;
```

```
public void setName(String name) {  
    this.name = name;  
}  
  
public String getName() {  
    return name;  
}  
}
```

Child:

```
class Dog extends Animal {  
  
    void display() {  
        System.out.println(getName());  
    }  
}
```

Notice:

Dog cannot directly do:

name

because name is private to Animal.

But Dog can use:

getName()

because that method is accessible.

This is an important benefit of encapsulation.

33. private does not mean "object doesn't have it"

This is a common misunderstanding.

Suppose:

```
class Student {
```



```
    private int age;  
}
```

The object still contains:

age

It is not deleted.

It is not invisible to Java.

It is simply **not directly accessible from outside the class**.

Think:

Object

```
|  
├── name  
├── age  
└── marks
```

but:

outside code

↓

cannot directly access private fields

The class itself can access them.

34. Encapsulation Example — Bank Account

Let's create a more realistic example.

```
class BankAccount {  
  
    private String accountHolder;  
    private double balance;  
  
    public void setAccountHolder(String accountHolder) {  
        this.accountHolder = accountHolder;  
    }  
}
```

```
public String getAccountHolder() {  
    return accountHolder;  
}
```

```
public void deposit(double amount) {  
  
    if (amount > 0) {  
        balance += amount;  
    }  
}
```

```
public void withdraw(double amount) {  
  
    if (amount > 0 && amount <= balance) {  
        balance -= amount;  
    } else {  
        System.out.println("Invalid withdrawal");  
    }  
}
```

```
public double getBalance() {  
    return balance;  
}  
}
```

Usage:

```
BankAccount account = new BankAccount();
```

```
account.setAccountHolder("Ravi");
```

```
account.deposit(5000);
```

```
account.withdraw(1000);
```

```
System.out.println(account.getAccountHolder());
```

```
System.out.println(account.getBalance());
```

Output:

Ravi

4000.0

Notice that there is **no**:

```
account.balance = 500000;
```

The balance is controlled by:

```
deposit()
```

```
withdraw()
```

This is better encapsulation than simply providing:

```
setBalance()
```

because a bank account shouldn't necessarily allow arbitrary balance replacement.

35. Important: Getter/Setter is not always enough

Suppose you write:

```
private double balance;
```

```
public void setBalance(double balance) {
```

```
    this.balance = balance;
```

```
}
```

Technically this is encapsulation-style code because the field is private and access is through a method.

But logically it may be a poor design.

Why?

Because now someone can do:

```
account.setBalance(-100000);
```

So you're hiding the variable, but you're not necessarily protecting the object's rules.

Better:

```
deposit()
```

```
withdraw()
```

with validation.

So remember:

Encapsulation is about controlled access, not blindly creating getters and setters for everything.

36. Access modifiers on methods too

Access modifiers aren't only for variables.

They can be used with:

Variables

```
private int age;
```

Methods

```
public void display() {  
}
```

Constructors

```
private Student() {  
}
```

Classes

```
public class Student {  
}
```

So access modifiers control the visibility/accessibility of different class members.

37. Public class example

```
public class Student {  
}
```

The class is public.

Other packages can access the class if it is available through the classpath/module structure.

38. Private methods

You can also have:

```
private void calculateMarks() {  
}
```

Outside code cannot call:

```
s.calculateMarks();
```

Why would we need this?

Sometimes a class has internal helper methods that should not be exposed.

Example:

```
class Calculator {  
  
    public void calculate() {  
        validate();  
        // calculation  
    }  
  
    private void validate() {  
        System.out.println("Validation");  
    }  
}
```

Outside code uses:

```
calculate()
```

but doesn't need to know about:

validate()

This is also part of controlling access.

39. Encapsulation vs Abstraction

You will encounter this question frequently.

They are related but different.

Encapsulation

Main idea:

How do we bundle and control access to data and behavior?

Example:

```
private int balance;
```

```
public double getBalance() {  
    return balance;  
}
```

Abstraction

Main idea:

What should the user see, and what implementation details should be hidden?

For example:

```
car.start();
```

You don't need to know all the internal engine operations.

Simple difference:

Encapsulation → protects/bundles data and behavior

Abstraction → hides unnecessary implementation complexity

Don't mix them.

40. Encapsulation vs Data Hiding

Again:

Data Hiding



Hide internal data



private

Whereas:

Encapsulation



Bundle data + methods

+

Control access

So:

Data hiding is part of encapsulation

but:

Encapsulation $\neq$ only data hiding

41. Encapsulation vs Access Modifiers

These are also different.

Access modifier

A Java language feature:

private

public

protected

Encapsulation

An OOP design principle.

We use access modifiers to help implement encapsulation.

Think:

Encapsulation



Need access control



Use access modifiers



private/public/protected/default

42. A very important real-world analogy

Imagine a **car**.

Internal data:

fuel

engineTemperature

engineSpeed

battery

You don't directly manipulate:

engineTemperature = 500

Instead you use controlled operations:

start()

accelerate()

brake()

Java equivalent:

```
class Car {
```

```
    private int engineTemperature;
```

```
    private int fuel;
```

```
    public void accelerate() {
```

```
        // controlled operation
```

```
    }
```



```
public void start() {  
    // controlled operation  
}  
}
```

Outside code interacts with the public interface.

Internal state stays protected.

43. The standard encapsulation pattern

When you see:

```
class Employee {  
  
    private String name;  
    private int age;  
    private double salary;  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    public int getAge() {  
        return age;  
    }  
  
    public void setAge(int age) {  
        this.age = age;  
    }  
}
```

```
}
```

```
public double getSalary() {  
    return salary;  
}
```

```
public void setSalary(double salary) {  
    this.salary = salary;  
}  
}
```

You should immediately recognize:

private variables

+

public getters/setters

=

encapsulation-style design

44. Why is this useful in large applications?

Imagine your project has:

100 classes

500 variables

1000 methods

If everything is public:

Class A can change Class B

Class B can change Class C

Class C can change Class D

...

It becomes difficult to understand who is changing what.

With encapsulation:

Class A



Public methods



Class B's internal state

Class B controls its own data.

This makes code easier to:

- maintain
 - debug
 - modify
 - validate
 - protect
 - understand
-

45. What happens when requirements change?

Suppose initially:

```
public void setAge(int age) {  
    this.age = age;  
}
```

Later requirement:

Age must be between 0 and 120.

You can change:

```
public void setAge(int age) {  
  
    if (age >= 0 && age <= 120) {  
        this.age = age;  
    }  
}
```

Outside code doesn't need to know how the internal implementation changed.

It still calls:

```
s.setAge(20);
```

This is another advantage of encapsulation.

46. Common Beginner Mistakes

Mistake 1

Thinking:

private means nobody can access it.

Not exactly.

Correct:

private means it cannot be directly accessed from outside its declaring class.

The class itself can access it.

Mistake 2

Thinking:

Every private variable needs getter and setter.

No.

You choose based on requirements.

Mistake 3

Thinking:

Getter changes the value.

No.

Usually:

Getter → reads

Setter → changes

Mistake 4

Thinking:

Setter is just another name for variable.

No.

Setter is a **method** that controls changing the variable.

Mistake 5

Thinking:

Encapsulation means making everything private.

Not exactly.

The idea is **controlled access**.

Some things should be public.

Some should be private.

Some may be protected/default depending on design.

47. One complete example

Let's put everything together.

```
class Student {  
  
    private String name;  
    private int age;  
    private int marks;  
  
    public Student(String name, int age, int marks) {  
        this.name = name;  
        this.age = age;  
        setMarks(marks);  
    }  
  
    public String getName() {  
        return name;  
    }  
}
```

```
}
```

```
public void setName(String name) {  
    this.name = name;  
}
```

```
public int getAge() {  
    return age;  
}
```

```
public void setAge(int age) {  
  
    if (age >= 0) {  
        this.age = age;  
    }  
}
```

```
public int getMarks() {  
    return marks;  
}
```

```
public void setMarks(int marks) {  
  
    if (marks >= 0 && marks <= 100) {  
        this.marks = marks;  
    } else {  
        System.out.println("Invalid marks");  
    }  
}
```

```
}
```

Usage:

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Student s = new Student("Ravi", 20, 85);
```

```
        System.out.println(s.getName());
```

```
        System.out.println(s.getAge());
```

```
        System.out.println(s.getMarks());
```

```
        s.setAge(21);
```

```
        s.setMarks(95);
```

```
        System.out.println(s.getAge());
```

```
        System.out.println(s.getMarks());
```

```
    }
```

```
}
```

Here we have:

Class

↓

Student

Private data

↓

name

age

marks

Constructor



Initializes object

Getters



Read data

Setters



Controlled modification

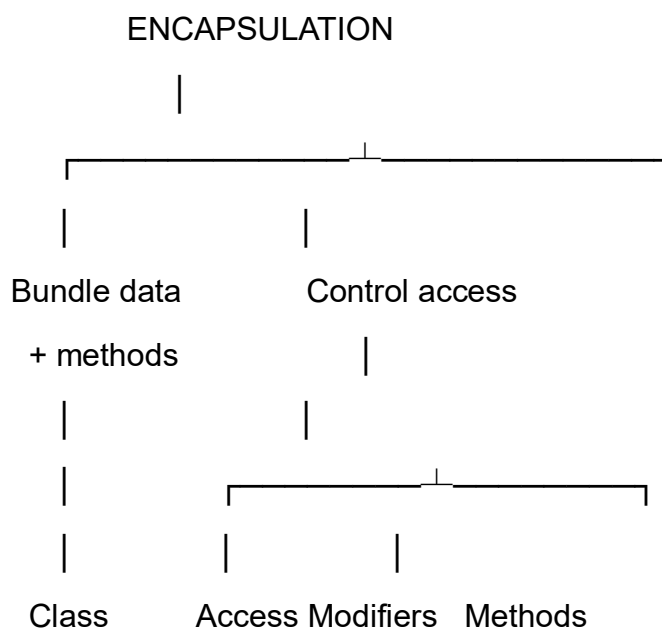
Validation

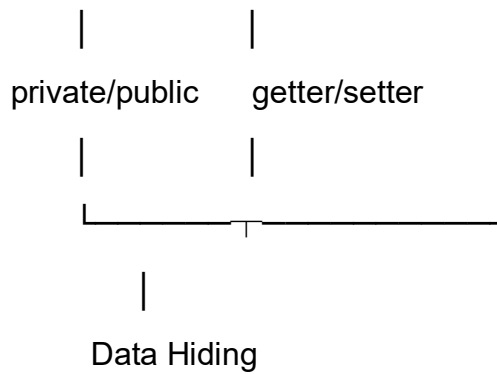


Prevents invalid values

That is a strong practical example of encapsulation.

48. The entire concept in one diagram





49. The most important mental model

Whenever you create a class, think:

"What data does this object own, and who should be allowed to change it?"

For example:

```
class BankAccount {
```

```
    private double balance;
```

```
}
```

Ask:

Can anyone directly change balance?

No.

Then:

How should balance be changed?

Through:

```
deposit()
```

```
withdraw()
```

Then:

How can someone see balance?

Through:

```
getBalance()
```

That is encapsulation thinking.

50. Quick revision

Encapsulation

Wrapping data + methods together

+

Controlling access to internal state

Access Modifier

Controls who can access something

Main ones:

private

default

protected

public

private

Accessible directly only inside the declaring class

Getter

```
public int getAge() {  
    return age;  
}
```

Used to **read**.

Setter

```
public void setAge(int age) {  
    this.age = age;  
}
```

Used to **change**.

Data hiding

Preventing direct access to internal data

Usually achieved using:

private

this

this.age = age;

means:

object's age = method parameter age

51. One-line differences you should remember

Concept	Meaning
Encapsulation	Bundle data + methods and control access
Data Hiding	Prevent direct access to internal data
Access Modifier	Defines accessibility
private	Direct access only inside declaring class
public	Broadly accessible
protected	Same package + subclass access rules
default	Same-package access
Getter	Reads a value
Setter	Changes a value
Validation	Checks whether incoming data is acceptable

52. Final mental picture

If you remember only one thing from today's lesson, remember this:

WITHOUT ENCAPSULATION

Outside

↓

Directly changes data

↓

Object

↓

No control

WITH ENCAPSULATION

Outside



Public method



Validation / Logic



Private data



Object remains controlled

For example:

```
class BankAccount {  
  
    private double balance;  
  
    public void deposit(double amount) {  
  
        if (amount > 0) {  
            balance += amount;  
        }  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
}
```

The important idea is **not** simply:

private + getter + setter

The deeper idea is:

"The object should control its own internal data instead of allowing outside code to freely manipulate it."